PYTHON NETWORKING

Python Socket Programming

A complete, minimal guide to the socket library from first byte to secure, concurrent servers

9	40+	TCP,
parts	tested examples	UDP, TLS, async

Every example in this guide was run and verified on Python 3.8+ (standard library only)

Contents

1. Foundations 3
2. The Socket API 5
3. TCP in Depth 8
4. UDP 12
5. Handling Many Clients 14
6. Options, Timeouts and Errors 17
7. Secure Sockets (TLS) 20
8. Practical Projects 22
9. Debugging and Cheat Sheet 26

1. Foundations

A **socket** is an endpoint for two-way communication between programs, on one machine or across a network. Python's `socket` module is a thin wrapper over the operating system's socket API (BSD sockets), so everything you learn here transfers directly to C, Go, Rust and Java.

2	1	0
main protocols (TCP, UDP)	import: socket	third-party packages

The mental model

Layer	What you control	Python tool
Application	Your message format (JSON, lines, binary)	`json`, `struct`
Transport	TCP (reliable stream) or UDP (datagrams)	`SOCK_STREAM`, `SOCK_DGRAM`
Network	IP address and routing	`AF_INET`, `AF_INET6`
Link	Ethernet, Wi-Fi	handled by the OS

Address families and socket types

Constant	Meaning
`AF_INET`	IPv4 address, written as `(host, port)`
`AF_INET6`	IPv6 address, written as `(host, port, flowinfo, scope_id)`
`AF_UNIX`	Local inter-process socket, address is a file path
`SOCK_STREAM`	TCP: ordered, reliable byte stream
`SOCK_DGRAM`	UDP: independent, unreliable messages
`SOCK_RAW`	Raw IP packets (needs root, advanced)

Client and server lifecycle

Step	Server (TCP)	Client (TCP)
1	`socket()` create	`socket()` create
2	`bind((host, port))` claim an address	
3	`listen()` start queueing connections	

Step	Server (TCP)	Client (TCP)
4	`accept()` returns a new socket per client	`connect((host, port))`
5	`recv()` / `sendall()`	`sendall()` / `recv()`
6	`close()` the client socket	`close()`

Key idea: The listening socket never carries data. `accept()` hands back a **new** socket dedicated to one client. The original keeps listening.

Bytes, not strings

Sockets move **bytes**. Convert explicitly at the boundary.

```
text = "héllo"
payload = text.encode("utf-8")     # str   -> bytes   (send this)
print(payload.decode("utf-8"))     # bytes -> str     (after recv)
```

Your first server and client

Start the server in one terminal, then run the client in another.

```
# server.py
import socket

HOST, PORT = "127.0.0.1", 5000

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind((HOST, PORT))
    srv.listen()
    print(f"listening on {HOST}:{PORT}")
    conn, addr = srv.accept()
    with conn:
        print("connected by", addr)
        while data := conn.recv(1024):     # b"" means the peer closed
            conn.sendall(data)             # echo back
```

```
# client.py
import socket

with socket.create_connection(("127.0.0.1", 5000)) as s:
    s.sendall(b"hello socket")
    print(s.recv(1024))                    # b'hello socket'
```

Tip: Always use sockets as context managers (`with`). They close automatically, even when an exception is raised.

2. The Socket API

Creating sockets

```python
import socket

tcp  = socket.socket(socket.AF_INET,  socket.SOCK_STREAM)   # IPv4 TCP
udp  = socket.socket(socket.AF_INET,  socket.SOCK_DGRAM)    # IPv4 UDP
tcp6 = socket.socket(socket.AF_INET6, socket.SOCK_STREAM)   # IPv6 TCP
unix = socket.socket(socket.AF_UNIX,  socket.SOCK_STREAM)   # local IPC

# Convenience helpers (Python 3.8+ for create_server)
client = socket.create_connection(("example.com", 80), timeout=5)
server = socket.create_server(("127.0.0.1", 5000))          # bind + listen + reuse addr
```

`create_connection` resolves the name, tries every address returned (IPv6 and IPv4) and applies the timeout. Prefer it to a manual `connect()` in client code.

Method reference

Method	Purpose
`bind(addr)`	Attach the socket to a local address
`listen(backlog)`	Mark as server; backlog is the pending-connection queue
`accept()`	Block until a client connects; returns `(conn, addr)`
`connect(addr)`	Open a connection to a server
`connect_ex(addr)`	Like `connect` but returns an error code instead of raising
`send(data)`	Send some bytes; returns how many were sent
`sendall(data)`	Send everything or raise (use this for TCP)
`recv(n)`	Receive up to `n` bytes; `b""` means closed
`recv_into(buf)`	Receive into a pre-allocated buffer, no extra copy
`sendto(data, addr)`	UDP: send one datagram
`recvfrom(n)`	UDP: receive `(data, sender_addr)`
`sendfile(file)`	Efficient file transmission (uses `sendfile(2)` when possible)
`shutdown(how)`	Close one direction: `SHUT_RD`, `SHUT_WR`, `SHUT_RDWR`
`close()`	Release the socket
`settimeout(s)`	`None` blocking, `0` non-blocking, `>0` timeout in seconds

Method	Purpose
`setblocking(flag)`	Shortcut for blocking / non-blocking mode
`getsockname()`	Local address of this socket
`getpeername()`	Remote address of the connected peer
`makefile(mode)`	Wrap the socket as a file object for line-based I/O
`fileno()`	Underlying file descriptor (used with `select`)

Name and address helpers

```
import socket

socket.gethostname()                    # this machine's name
socket.gethostbyname("localhost")       # '127.0.0.1'   (IPv4 only)
socket.gethostbyaddr("127.0.0.1")       # reverse lookup
socket.getfqdn()                        # fully qualified domain name

socket.inet_aton("192.168.1.10")        # b'\xc0\xa8\x01\n'  (4 bytes)
socket.inet_ntoa(b"\xc0\xa8\x01\n")     # '192.168.1.10'
socket.inet_pton(socket.AF_INET6, "::1")  # 16 bytes, works for IPv6 too
socket.inet_ntop(socket.AF_INET6, b"\x00" * 15 + b"\x01")   # '::1'

socket.getservbyname("http")            # 80
socket.getservbyport(443)               # 'https'
```

Byte order

Network protocols use **big-endian** ("network byte order"). Convert integers with `htons` / `ntohs`, or better, with `struct` and the `!` prefix.

```
import socket, struct

socket.htons(80)                # 20480  (host -> network, 16-bit)
socket.ntohs(20480)             # 80

header = struct.pack("!HHI", 1, 80, 1_000_000)   # 2 + 2 + 4 bytes, big-endian
a, b, c = struct.unpack("!HHI", header)
```

getaddrinfo: the modern way to resolve

`getaddrinfo` returns every usable address for a name, for any family, and is the correct basis for IPv6-ready code.

```python
import socket

for family, type_, proto, canon, sockaddr in socket.getaddrinfo(
        "localhost", 80, type=socket.SOCK_STREAM):
    print(family.name, sockaddr)
# AF_INET ('127.0.0.1', 80)
```

IPv6 and dual-stack servers

```python
import socket

# One server socket that accepts both IPv4 and IPv6 clients
if socket.has_dualstack_ipv6():
    srv = socket.create_server(("::", 5000),
                               family=socket.AF_INET6,
                               dualstack_ipv6=True)
```

Special addresses: `127.0.0.1` / `::1` loopback (this machine only). `0.0.0.0` / `::` all interfaces. `""` also means all interfaces in `bind()`. Port `0` lets the OS choose a free port; read it back with `getsockname()`.

3. TCP in Depth

TCP is a stream, not messages

This is the most important concept in socket programming. TCP guarantees that bytes arrive **in order and complete**, but it does **not** preserve the boundaries of your `send()` calls.

You send	The receiver may get
`send(b"HELLO")`, `send(b"WORLD")`	`b"HELLOWORLD"` in one `recv`
`send(b"HELLOWORLD")`	`b"HELLOWO"` then `b"RLD"`

So every real protocol needs **framing**: a rule telling the receiver where one message ends. Three standard solutions follow.

Framing 1: delimiter (lines)

The simplest approach: end each message with `\n`. Use `makefile()` so Python handles the buffering.

```
# line server: uppercases each line
import socket

with socket.create_server(("127.0.0.1", 5000)) as srv:
    conn, _ = srv.accept()
    with conn, conn.makefile("rwb") as f:
        for line in f:                # yields one line at a time
            f.write(line.upper())
            f.flush()
```

Best for text protocols (HTTP headers, SMTP, Redis-style commands). Weak for binary data that may contain the delimiter.

Framing 2: length prefix

Send a fixed-size length header, then exactly that many bytes. This is the most robust approach for arbitrary data.

```python
import socket, struct

def recv_exact(sock, n):
    """Read exactly n bytes or raise if the peer disconnects."""
    buf = bytearray()
    while len(buf) < n:
        chunk = sock.recv(n - len(buf))
        if not chunk:
            raise ConnectionError("peer closed connection")
        buf.extend(chunk)
    return bytes(buf)

def send_msg(sock, payload: bytes):
    sock.sendall(struct.pack("!I", len(payload)) + payload)   # 4-byte length

def recv_msg(sock, max_size=1_048_576):
    (length,) = struct.unpack("!I", recv_exact(sock, 4))
    if length > max_size:                                     # never trust the peer
        raise ValueError("message too large")
    return recv_exact(sock, length)
```

Always cap the length: Without `max_size`, a malicious peer can announce a 4 GB message and exhaust your memory. Validate every length field before allocating.

Framing 3: JSON over length prefix

Build structured messages on top of the framing helpers above.

```python
import json

def send_json(sock, obj):
    send_msg(sock, json.dumps(obj).encode("utf-8"))

def recv_json(sock):
    return json.loads(recv_msg(sock).decode("utf-8"))

# server side
def serve(conn):
    while True:
        try:
            request = recv_json(conn)
        except ConnectionError:
            break
        send_json(conn, {"echo": request, "keys": len(request)})

# client side
send_json(sock, {"cmd": "ping", "id": 7})
print(recv_json(sock))      # {'echo': {'cmd': 'ping', 'id': 7}, 'keys': 2}
```

Choosing a framing strategy

Strategy	Strength	Weakness	Use for
Delimiter	Human readable, trivial	Delimiter must not appear in data	Chat, commands, logs
Length prefix	Binary safe, fast	Not readable in a terminal	File transfer, RPC, games
Fixed size	Zero parsing	Wastes space	Tiny control messages
Close to end	No header at all	One message per connection	Simple downloads

recv() behaviour you must know

Situation	Result
Data available	Returns between 1 and `n` bytes
Peer closed cleanly	Returns `b""` (empty bytes)
Nothing yet, blocking socket	Waits
Nothing yet, non-blocking socket	Raises `BlockingIOError`
Timeout expired	Raises `TimeoutError` (alias `socket.timeout`)
Peer crashed or reset	Raises `ConnectionResetError`

`send()` can also be partial. Use `sendall()` for TCP unless you are writing a non-blocking event loop.

Half-close: finish sending, keep receiving

`shutdown(SHUT_WR)` tells the peer "I am done sending" while you can still read the reply. The peer sees `b""` from `recv`.

```python
# client
import socket

with socket.create_connection(("127.0.0.1", 5000)) as s:
    s.sendall(b"x" * 100)
    s.shutdown(socket.SHUT_WR)        # send EOF, keep the read side open
    print(s.recv(100))                # b'got 100 bytes'

# server side: read until EOF, then answer
data = b""
while chunk := conn.recv(1024):
    data += chunk
conn.sendall(b"got %d bytes" % len(data))
```

4. UDP

UDP is connectionless: no handshake, no stream, no delivery guarantee. Each `sendto` is one independent **datagram**, and each `recvfrom` returns exactly one.

Property	TCP	UDP
Connection	Yes (`connect` / `accept`)	No
Reliability	Retransmits lost data	None, packets may vanish
Ordering	Guaranteed	Not guaranteed
Message boundaries	No (stream)	Yes (datagram)
Overhead	Higher	Minimal
Typical use	Web, email, SSH, APIs	DNS, VoIP, games, discovery, telemetry

UDP echo server and client

```python
# udp_server.py
import socket

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
    s.bind(("127.0.0.1", 9999))
    while True:
        data, addr = s.recvfrom(2048)       # one whole datagram
        s.sendto(data.upper(), addr)        # reply to whoever sent it
```

```python
# udp_client.py
import socket

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
    s.settimeout(2)                         # UDP can silently lose packets
    s.sendto(b"ping", ("127.0.0.1", 9999))
    try:
        data, server = s.recvfrom(2048)
        print(data, server)
    except socket.timeout:
        print("no reply")
```

Always set a timeout on UDP clients: A lost datagram means `recvfrom` would wait forever. Add timeouts and, if you need reliability, your own retry and sequence numbers.

Broadcast

Send one datagram to every host on the local network.

```python
import socket

s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
s.setsockopt(socket.SOL_SOCKET, socket.SO_BROADCAST, 1)   # required
s.sendto(b"anyone here?", ("255.255.255.255", 9999))
```

Multicast

Send to a **group** address (224.0.0.0 to 239.255.255.255). Only hosts that joined the group receive it.

```python
# receiver
import socket, struct

GROUP, PORT = "224.1.1.1", 5007
r = socket.socket(socket.AF_INET, socket.SOCK_DGRAM, socket.IPPROTO_UDP)
r.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
r.bind(("", PORT))
mreq = struct.pack("4s4s", socket.inet_aton(GROUP), socket.inet_aton("0.0.0.0"))
r.setsockopt(socket.IPPROTO_IP, socket.IP_ADD_MEMBERSHIP, mreq)
print(r.recvfrom(1024))
```

```python
# sender
import socket

snd = socket.socket(socket.AF_INET, socket.SOCK_DGRAM, socket.IPPROTO_UDP)
snd.setsockopt(socket.IPPROTO_IP, socket.IP_MULTICAST_TTL, 2)   # hop limit
snd.sendto(b"hello group", ("224.1.1.1", 5007))
```

Connected UDP

Calling `connect()` on a UDP socket does not open a connection. It only fixes the default peer, so you can use `send` / `recv` and the kernel filters out packets from other senders.

```python
with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
    s.connect(("127.0.0.1", 9999))
    s.send(b"hi")
    print(s.recv(1024))
```

5. Handling Many Clients

A blocking `accept()` / `recv()` loop serves one client at a time. Four standard models solve this.

Model	How it scales	Complexity	Best for
Thread per client	Tens to hundreds of clients	Low	Learning, simple tools
`selectors` event loop	Thousands of clients	Medium	Custom protocols, one thread
`socketserver`	Same as threads	Lowest	Quick prototypes
`asyncio`	Thousands of clients	Medium	Modern network apps

Model 1: a thread per client

```
import socket, threading

def handle(conn, addr):
    with conn:
        while data := conn.recv(1024):
            conn.sendall(data)

with socket.create_server(("127.0.0.1", 5000)) as srv:
    while True:
        conn, addr = srv.accept()
        threading.Thread(target=handle, args=(conn, addr), daemon=True).start()
```

Model 2: selectors (single-threaded event loop)

One thread watches many non-blocking sockets and reacts only when one is ready. The callback is stored in `key.data`.

```python
import selectors, socket

sel = selectors.DefaultSelector()

def accept(srv):
    conn, addr = srv.accept()
    conn.setblocking(False)
    sel.register(conn, selectors.EVENT_READ, echo)

def echo(conn):
    data = conn.recv(1024)
    if data:
        conn.send(data)           # simplified: may send only part of data
    else:
        sel.unregister(conn)
        conn.close()

srv = socket.create_server(("127.0.0.1", 5000))
srv.setblocking(False)
sel.register(srv, selectors.EVENT_READ, accept)

while True:
    for key, _ in sel.select():
        key.data(key.fileobj)     # call the registered handler
```

Production note: A real event loop keeps a per-connection output buffer and registers `EVENT_WRITE` when `send` could not flush everything. `asyncio` does this for you.

Model 3: socketserver

The standard library's ready-made server framework. You only write the handler.

```python
import socketserver

socketserver.ThreadingTCPServer.allow_reuse_address = True

class Handler(socketserver.StreamRequestHandler):
    def handle(self):
        for line in self.rfile:              # self.rfile / self.wfile are file objects
            self.wfile.write(line.upper())

with socketserver.ThreadingTCPServer(("127.0.0.1", 5000), Handler) as srv:
    srv.serve_forever()
```

Model 4: asyncio streams

```python
# server
import asyncio

async def handle(reader, writer):
    while data := await reader.read(1024):
        writer.write(data)
        await writer.drain()                # respect back-pressure
    writer.close()
    await writer.wait_closed()

async def main():
    server = await asyncio.start_server(handle, "127.0.0.1", 5000)
    async with server:
        await server.serve_forever()

asyncio.run(main())
```

```python
# client
import asyncio

async def main():
    reader, writer = await asyncio.open_connection("127.0.0.1", 5000)
    writer.write(b"ping")
    await writer.drain()
    print(await reader.read(100))
    writer.close()
    await writer.wait_closed()

asyncio.run(main())
```

select(): the classic primitive

`selectors` is a portable wrapper. The lower-level `select.select(rlist, wlist, xlist, timeout)` is still useful for a quick "is anything readable?" check.

```python
import select

readable, _, _ = select.select([sock], [], [], 1.0)   # wait up to 1 second
if readable:
    data = sock.recv(1024)
```

6. Options, Timeouts and Errors

Socket options

`setsockopt(level, option, value)` tunes socket behaviour.

Option	Level	Effect
`SO_REUSEADDR`	`SOL_SOCKET`	Rebind quickly after restart (avoids "Address already in use")
`SO_REUSEPORT`	`SOL_SOCKET`	Several sockets share one port (Linux, BSD)
`SO_KEEPALIVE`	`SOL_SOCKET`	Probe idle connections to detect dead peers
`SO_RCVBUF` / `SO_SNDBUF`	`SOL_SOCKET`	Kernel buffer sizes
`SO_BROADCAST`	`SOL_SOCKET`	Allow UDP broadcast
`SO_LINGER`	`SOL_SOCKET`	Behaviour of `close()` with unsent data
`TCP_NODELAY`	`IPPROTO_TCP`	Disable Nagle's algorithm for low latency
`IP_MULTICAST_TTL`	`IPPROTO_IP`	Multicast hop limit

```python
import socket, struct

s = socket.socket()
s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
s.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1)
s.setsockopt(socket.SOL_SOCKET, socket.SO_KEEPALIVE, 1)

print(s.getsockopt(socket.SOL_SOCKET, socket.SO_RCVBUF))      # read an option

# Linger on: close() blocks up to 1 s;  (1, 0) would send RST immediately
s.setsockopt(socket.SOL_SOCKET, socket.SO_LINGER, struct.pack("ii", 1, 5))
```

Blocking modes and timeouts

Call	Mode	Behaviour
`s.settimeout(None)`	Blocking	Wait forever (default)
`s.settimeout(5.0)`	Timeout	Raise `TimeoutError` after 5 s
`s.settimeout(0)` / `s.setblocking(False)`	Non-blocking	Raise `BlockingIOError` if not ready
`socket.setdefaulttimeout(5)`	Global	Default for sockets created afterwards

```python
import socket

try:
    with socket.create_connection(("10.255.255.1", 80), timeout=3) as s:
        s.settimeout(5)                    # separate timeout for I/O
        s.sendall(b"ping")
        print(s.recv(1024))
except socket.timeout:
    print("timed out")
except ConnectionRefusedError:
    print("nothing is listening there")
```

Exception hierarchy

All socket errors derive from `OSError`. Catch the specific subclass first.

Exception	Typical cause
`ConnectionRefusedError`	No server listening on that port
`ConnectionResetError`	Peer sent RST or crashed
`ConnectionAbortedError`	Connection dropped locally
`BrokenPipeError`	Writing after the peer closed
`TimeoutError` (`socket.timeout`)	Operation exceeded its timeout
`BlockingIOError`	Non-blocking socket not ready
`socket.gaierror`	Name resolution failed (DNS)
`socket.herror`	Reverse lookup error
`OSError: [Errno 98] Address already in use`	Port busy or in `TIME_WAIT`
`PermissionError`	Ports below 1024 need privileges

```python
import socket

try:
    socket.getaddrinfo("no-such-host.invalid", 80)
except socket.gaierror as e:
    print("DNS failure:", e)
```

A robust connect-with-retry helper

```python
import socket, time

def connect_retry(host, port, attempts=5, delay=0.5, timeout=3):
    for i in range(attempts):
        try:
            return socket.create_connection((host, port), timeout=timeout)
        except (ConnectionRefusedError, socket.timeout, socket.gaierror) as e:
            if i == attempts - 1:
                raise
            time.sleep(delay * 2 ** i)      # exponential back-off
```

7. Secure Sockets (TLS)

Plain sockets are readable and modifiable by anyone on the path. The `ssl` module wraps a socket with **TLS** for encryption, integrity and server authentication.

TLS client

```python
import socket, ssl

ctx = ssl.create_default_context()          # verifies certificates and hostname
host = "example.com"

with socket.create_connection((host, 443), timeout=5) as raw:
    with ctx.wrap_socket(raw, server_hostname=host) as tls:
        print(tls.version(), tls.cipher()[0])       # e.g. TLSv1.3 TLS_AES_256_GCM_SHA384
        tls.sendall(f"HEAD / HTTP/1.1\r\nHost: {host}\r\n"
                    f"Connection: close\r\n\r\n".encode())
        print(tls.recv(4096).decode(errors="replace"))
```

Never disable verification: `ctx.check_hostname = False` or `ctx.verify_mode = ssl.CERT_NONE` removes the protection TLS exists to give you. If you need a private CA or a self-signed cert, load it with `load_verify_locations` instead.

Create a self-signed certificate (for testing)

```bash
openssl req -x509 -newkey rsa:2048 -nodes -days 365 \
  -keyout key.pem -out cert.pem \
  -subj "/CN=localhost" \
  -addext "subjectAltName=DNS:localhost,IP:127.0.0.1"
```

TLS server

```python
import socket, ssl

ctx = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
ctx.minimum_version = ssl.TLSVersion.TLSv1_2
ctx.load_cert_chain("cert.pem", "key.pem")

with socket.create_server(("127.0.0.1", 8443)) as srv:
    with ctx.wrap_socket(srv, server_side=True) as tsrv:
        conn, addr = tsrv.accept()          # TLS handshake happens here
        with conn:
            conn.sendall(conn.recv(1024).upper())
```

TLS client trusting your own certificate

```python
import socket, ssl

ctx = ssl.create_default_context(cafile="cert.pem")     # trust only this cert

with socket.create_connection(("127.0.0.1", 8443)) as raw:
    with ctx.wrap_socket(raw, server_hostname="localhost") as tls:
        tls.sendall(b"secret")
        print(tls.recv(100))                            # b'SECRET'
        print(tls.getpeercert()["subject"])
```

Mutual TLS (client certificates)

```python
# server: demand a client certificate signed by your CA
ctx.verify_mode = ssl.CERT_REQUIRED
ctx.load_verify_locations("client_ca.pem")

# client: present its own certificate
ctx.load_cert_chain("client.pem", "client.key")
```

Security checklist for socket code

- [ ] Bind to `127.0.0.1` unless the service must be reachable from the network
- [ ] Set a timeout on every socket; a silent peer must never hang you
- [ ] Cap every length field, buffer and line length before allocating
- [ ] Never `pickle.loads()`, `eval()` or `exec()` data that came from a socket
- [ ] Use TLS for anything crossing an untrusted network; keep verification on
- [ ] Sanitise file names, paths and commands built from network input
- [ ] Limit concurrent connections and rate per client to resist resource exhaustion
- [ ] Handle `ConnectionResetError` and half-open connections without crashing
- [ ] Log peer addresses and errors, never secrets or session tokens

8. Practical Projects

HTTP request over a raw socket

HTTP/1.1 is just text over TCP. This is how `requests` works at the lowest level.

```python
import socket

host = "example.com"
req = f"GET / HTTP/1.1\r\nHost: {host}\r\nConnection: close\r\n\r\n".encode()

with socket.create_connection((host, 80), timeout=5) as s:
    s.sendall(req)
    chunks = []
    while chunk := s.recv(4096):            # read until the server closes
        chunks.append(chunk)

head, _, body = b"".join(chunks).partition(b"\r\n\r\n")
print(head.decode().splitlines()[0])        # HTTP/1.1 200 OK
```

TCP port checker and scanner

A connect scan simply tries to open a connection. `connect_ex` returns `0` on success.

```python
import socket
from concurrent.futures import ThreadPoolExecutor

def check(host, port, timeout=0.5):
    with socket.socket() as s:
        s.settimeout(timeout)
        return port if s.connect_ex((host, port)) == 0 else None

host = "127.0.0.1"
with ThreadPoolExecutor(100) as pool:
    found = pool.map(lambda p: check(host, p), range(1, 1025))
    print("open:", [p for p in found if p])
```

> **Authorization:** Scan only systems you own or have written permission to test. Unauthorised scanning can be illegal and is easily detected.

Banner grabbing

Many services announce themselves on connect (SSH, FTP, SMTP). The banner is useful for inventory and for checking your own exposure.

```python
import socket

def grab_banner(host, port, timeout=2):
    with socket.create_connection((host, port), timeout=timeout) as s:
        try:
            return s.recv(256).decode(errors="replace").strip()
        except socket.timeout:
            return ""                       # service waits for us to speak first

print(grab_banner("127.0.0.1", 22))         # e.g. SSH-2.0-OpenSSH_9.6
```

File transfer with a header

A small binary protocol: 2-byte name length, 8-byte file size, name, then raw content. It reuses `recv_exact` from Part 3.

```python
import os, socket, struct

def send_file(path, host, port):
    name = os.path.basename(path).encode()
    size = os.path.getsize(path)
    with socket.create_connection((host, port)) as s, open(path, "rb") as f:
        s.sendall(struct.pack("!HQ", len(name), size) + name)
        s.sendfile(f)                                   # zero-copy where supported

def receive_file(conn, out_dir="."):
    nlen, size = struct.unpack("!HQ", recv_exact(conn, 10))
    name = os.path.basename(recv_exact(conn, nlen).decode())   # strip any path parts
    if name in ("", ".", ".."):
        raise ValueError("bad file name")
    with open(os.path.join(out_dir, name), "wb") as f:
        remaining = size
        while remaining:
            chunk = conn.recv(min(65536, remaining))
            if not chunk:
                raise ConnectionError("transfer truncated")
            f.write(chunk)
            remaining -= len(chunk)
```

Why basename()?: A hostile sender could use the name `../../etc/cron.d/x`. Stripping directory parts prevents path traversal. Also enforce a maximum `size`.

Multi-user chat server

One thread, many clients, using `selectors`. Connect with `nc 127.0.0.1 6000` from several terminals.

```python
import selectors, socket

sel = selectors.DefaultSelector()
clients = {}                                # socket -> display name

def broadcast(msg, exclude=None):
    for c in list(clients):
        if c is not exclude:
            try:
                c.send(msg)
            except OSError:
                drop(c)

def drop(conn):
    name = clients.pop(conn, None)
    if name is None:
        return
    sel.unregister(conn)
    conn.close()
    broadcast(f"* {name} left\n".encode())

def accept(srv):
    conn, addr = srv.accept()
    conn.setblocking(False)
    clients[conn] = f"{addr[0]}:{addr[1]}"
    sel.register(conn, selectors.EVENT_READ)
    broadcast(f"* {clients[conn]} joined\n".encode(), exclude=conn)

srv = socket.create_server(("127.0.0.1", 6000))
srv.setblocking(False)
sel.register(srv, selectors.EVENT_READ)

while True:
    for key, _ in sel.select():
        sock = key.fileobj
        if sock is srv:
            accept(srv)
            continue
        try:
            data = sock.recv(1024)
        except OSError:
            data = b""
        if data:
            broadcast(f"{clients[sock]}: ".encode() + data, exclude=sock)
        else:
            drop(sock)
```

Unix domain sockets

Same API, but the address is a file path. Faster than TCP on loopback and protected by file permissions.

```python
# server
import os, socket

PATH = "/tmp/demo.sock"
if os.path.exists(PATH):
    os.unlink(PATH)

with socket.socket(socket.AF_UNIX, socket.SOCK_STREAM) as srv:
    srv.bind(PATH)
    srv.listen()
    conn, _ = srv.accept()
    with conn:
        conn.sendall(conn.recv(1024)[::-1])      # reverse the message
```

```python
# client
import socket

with socket.socket(socket.AF_UNIX, socket.SOCK_STREAM) as s:
    s.connect("/tmp/demo.sock")
    s.sendall(b"hello")
    print(s.recv(100))                           # b'olleh'
```

socketpair: a connected pair in one call

Ideal for talking between threads or between a parent and a child process, and for waking an event loop.

```python
import socket

a, b = socket.socketpair()
a.sendall(b"ping")
print(b.recv(4))                                 # b'ping'
a.close(); b.close()
```

9. Debugging and Cheat Sheet

Command-line tools

Goal	Command
List listening TCP sockets with process	`ss -tlnp`
Who uses port 5000?	`lsof -i :5000`
Quick TCP server for testing	`nc -l 5000`
Quick TCP client	`nc 127.0.0.1 5000`
Quick UDP listener	`nc -ul 9999`
Capture traffic on loopback	`sudo tcpdump -i lo -A port 5000`
Inspect TLS handshake	`openssl s_client -connect host:443`
Graphical packet analysis	Wireshark

Common problems

Symptom	Cause and fix
`Address already in use`	Previous run in `TIME_WAIT`; set `SO_REUSEADDR` or use `create_server`
`recv` returns partial messages	TCP has no boundaries; add framing and `recv_exact`
`recv` returns `b""` in a loop	Peer closed; stop reading and close your side
Client hangs forever	No timeout set; use `settimeout` and `create_connection(timeout=...)`
`Connection refused`	Server not running, wrong port, or bound to a different interface
Works locally, not remotely	Server bound to `127.0.0.1`; bind to `0.0.0.0` and open the firewall
`BrokenPipeError` on send	Peer already closed; catch it and clean up
Messages glued together	Two sends merged; frame your protocol
UDP packets missing	Normal for UDP; add acknowledgements or switch to TCP
`TypeError: a bytes-like object is required`	You passed `str`; call `.encode()`

One-page cheat sheet

Task	Code
TCP server	`s = socket.create_server((host, port))`
Accept a client	`conn, addr = s.accept()`
TCP client	`s = socket.create_connection((host, port), timeout=5)`
Send all bytes	`s.sendall(b"data")`
Receive	`data = s.recv(4096)`
UDP send	`s.sendto(b"data", (host, port))`
UDP receive	`data, addr = s.recvfrom(2048)`
Reuse address	`s.setsockopt(SOL_SOCKET, SO_REUSEADDR, 1)`
Timeout	`s.settimeout(5)`
Line I/O	`f = s.makefile("rwb")`
Frame header	`struct.pack("!I", len(msg))`
Wrap in TLS	`ssl.create_default_context().wrap_socket(s, server_hostname=h)`
Event loop	`selectors.DefaultSelector()`
Async	`asyncio.start_server` / `asyncio.open_connection`

Learning path

Build next

1. A key-value store with a text protocol
2. A TLS-protected chat with usernames
3. A TCP proxy that forwards and logs traffic
4. A UDP service-discovery beacon
5. A mini HTTP server on raw sockets

Study next

1. The `asyncio` protocols and transports API
2. `struct` and binary protocol design
3. TCP states: SYN, ESTABLISHED, TIME_WAIT
4. Scapy for packet crafting (lab use)
5. The Python docs: `socket`, `ssl`, `selectors`

> **Final advice:** Design the **protocol** first (framing, errors, limits), then write the code. Most socket bugs are protocol bugs.